Supplementary Information for "A two-ratio framework separates residency and transfer overlap in hierarchical-memory inference"

Geunsik Lim[1*]

[1]College of Information and Communication Engineering, Sungkyunkwan University (SKKU), Suwon, South Korea.

Corresponding author(s). E-mail(s): leemgs@g.skku.edu;

This document provides derivations, data provenance, reproduction commands, and the protocol boundary accompanying the main text. Equation numbers of the form (3) refer to the main text; equations numbered (S1), (S2), ... are internal to this document.

Supplementary Note 1 Derivation and implementation details

1.1 Overlap boundary

With compulsory transfer volume D and sustained link bandwidth B, the transfer time is $T_{\text{transfer}} = D/B$. Perfect overlap can hide at most T_{comp}, leaving

$$T_{\text{exposed}} \geq \max(0, T_{\text{transfer}} - T_{\text{comp}}). \tag{S1}$$

Thus $R_B = T_{\text{comp}}/T_{\text{transfer}} = 1$ is the equality boundary. This is a definition-derived boundary, not a fitted constant.

The manuscript uses a maximum, rather than a sum, for the service-time lower bound. This permits perfect overlap among compute, memory service, and transfer and avoids counting the same data movement twice. Any synchronisation, contention, or incomplete overlap increases latency above that conservative bound.

1.2 Residency convention

The non-resident fraction under the aggregate-capacity approximation is $\max(0, 1 - R_C)$. We label $R_C < 0.5$ capacity-limited because more than half of the working set is then non-resident. The choice is transparent and replaceable; it does not imply a singularity at 0.5.

1.3 Complete CPU data

The machine-readable files `code/results/cpu_probe/cpu_hierarchy_records.jsonl` and `code/results/cpu_probe/cpu_hierarchy_summary.json` contain all 14 pointer-chain points and seven-trial summaries. The files `code/results/colab_probe/records.jsonl` and `code/results/colab_probe/summary.json` contain the layered-matrix CPU measurements, and the device-tagged files `accel_tesla-t4.json`, `accel_a100.json`, and `accel_tpu-xla.json` (same directory) contain the per-point accelerator measurements for the NVIDIA Tesla T4, the NVIDIA A100, and the Google TPU v5e. Values in the main text are direct rounded summaries of those files.

1.4 Reproduction commands

From the repository root, the CPU measurements can be repeated with

```
python code/experiments/cpu_hierarchy_probe.py
python code/experiments/colab_regime_measurement.py \
  --backend numpy --quick
python code/experiments/reproduce_regime_map.py
```

The GPU measurements are reproduced by running the same harness on a CUDA device (e.g. a free cloud GPU runtime); it auto-detects the accelerator:

```
python code/experiments/colab_regime_measurement.py
```

The `reproduce_regime_map.py` command regenerates an analytical schematic and does not perform an experiment. Measured latencies vary with processor, GPU, BLAS/CUDA library, system load, and memory placement; exact numeric identity is not expected on other machines.

1.5 Accelerator validation status

Runs reported in the main-text accelerator Results were performed on three accelerators of two vendors: an NVIDIA Tesla T4 and an NVIDIA A100 through the CUDA-event path, and a Google TPU v5e through the `torch_xla` path. The residency direction is descriptively consistent on all three and the T4 sweep reaches both sides of the overlap coordinate; these are proof-of-concept checks, not a preregistered multi-platform campaign. Such a campaign should retain raw device-event timestamps, software versions, accelerator identifiers, warm-up policy, excluded-window reasons,

and every prespecified operating point, and should enlarge the per-step workload so that per-op timing is not dominated by launch overhead on fast devices.